9: Service discovery deep dive

In this chapter, you'll learn what service discovery is, why it's important, and how it's implemented in Kubernetes. You'll also learn some troubleshooting tips.

To get the most from this chapter, you should know what Kubernetes Services are and how they work.

The chapter is split into the following sections:

- Quick background
- Service registration
- Service discovery
- Service discovery and Namespaces
- Troubleshooting service discovery

> **Note:** The word "service" has a lot of meanings. So for clarity, we capitalise the first letter when referring to the Service resource in the Kubernetes API.

Quick background

Here's the outrageously high-level… finding stuff on a crazy-busy platform like Kubernetes is hard. Service discovery makes it possible.

Let's paint a bit more of a picture.

Kubernetes runs *cloud-native microservices* apps that scale up and down, self-heal from failures, and regularly get replaced by newer releases. All of this makes individual app instances unreliable. To solve this, Kubernetes has a super-stable Service object that fronts unreliable app instances with a stable IP, DNS name, and port. All good so far, but in a big bustling environment like many Kubernetes clusters, apps need a way to find the other apps they work with. This is where *service discovery* comes into play.

There are two major components to service discovery:

- Registration
- Discovery

Service registration

Service registration is the process of an application posting its connection details to a *service registry* so other apps can find it and consume it.

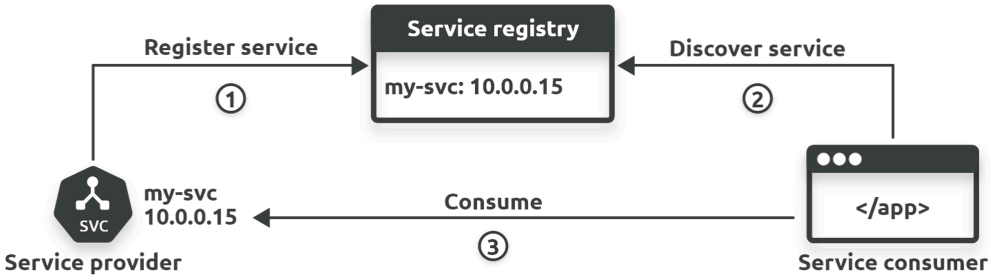

Figure 9.1

A few important things to note about service discovery in Kubernetes:

1. Kubernetes uses its internal DNS as a *service registry*
2. All Kubernetes Services automatically register their details with DNS

For this to work, Kubernetes provides a *well-known* internal DNS service that we usually call the "cluster DNS". It's *well known* because every Pod in the cluster knows where to find it. It's implemented in the `kube-system` Namespace as a set of Pods managed by a Deployment called `coredns`. These Pods are fronted by a Service called `kube-dns`. Behind the scenes, it's based on a DNS technology called CoreDNS and runs as a *Kubernetes-native application.*

The previous sentence contains a lot of detail, so the following commands show how its implemented. You can run them on your own Kubernetes cluster.

This command lists the Pods running the cluster DNS.

```
$ kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS   AGE
coredns-5644d7b6d9-fk4c9   1/1     Running   0          28d
coredns-5644d7b6d9-s5zlr   1/1     Running   0          28d
```

This lists the Deployment managing them.

```
$ kubectl get deploy -n kube-system -l k8s-app=kube-dns
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
coredns   2/2     2            2           28d
```

This lists the Service fronting them. It shows the *well known* IP configured on every Pod/container.

```
$ kubectl get svc -n kube-system -l k8s-app=kube-dns
NAME       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   192.168.200.10   <none>        53/UDP,53/TCP,9153/TCP   28d
```

The process of service registration looks like this (exact flow might slightly differ):

1. You post a new Service manifest to the API server
2. The request is authenticated, authorized, and subjected to admission policies
3. The Service is allocated a stable virtual IP address called a *ClusterIP*
4. An Endpoints object (or EndpointSlice) is created to hold a list of healthy Pods matching the Service's label selector
5. The *Pod network* is configured to handle traffic sent to the ClusterIP (more on this later)
6. The Service's name and IP are registered with the cluster DNS

Step 6 is the secret sauce.

We mentioned earlier that cluster DNS is a *Kubernetes-native application.* This means it knows it's running on Kubernetes and implements a controller that watches the API server for new Service objects. Any time it observes one, it automatically creates the DNS records mapping the Service name to its ClusterIP. This means apps, and even Services, don't need to perform their own service registration – the cluster DNS does it for them.

It's important to understand that the name registered in DNS for the Service is the value stored in its `metadata.name` property. This is why it's important that Service names are valid DNS names and don't include exotic characters. The ClusterIP is dynamically assigned by Kubernetes.

```
apiVersion: v1
kind: Service
metadata:
  name: ent      <<=== this name is registered with the cluster DNS
spec:
  selector:
    app: web
  ports:
    ...
```

At this point, the Service's *front-end* configuration (name, IP, port) is registered with DNS and the Service can be discovered by apps and clients.

The Service back-end

Now the Service's front-end is registered and can be discovered by other apps, the back-end needs building so there's something to send traffic to. This involves maintaining a list of healthy Pod IPs the Service will load-balance traffic to.

As explained in the chapter on Services, every Service has a `label selector` that determines which Pods it will load-balance traffic to. See Figure 9.2.

Figure 9.2

To help with backend operations, such as knowing which Pods to send traffic to and how traffic is routed, Kubernetes builds and Endpoints/EndpointSlice for every Service.

The following command shows an Endpoints object for a Service called `ent`. It has the IP address and port of two Pods matching the corresponding Service's label selector.

```
$ kubectl get endpoint ent
NAME    ENDPOINTS                                     AGE
ent     192.168.129.46:8080,192.168.130.127:8080      14m
```

Figure 9.3 shows the same `ent` Service that will load-balance traffic to two Pods. It also shows the Endpoints object with the IPs of the two Pods matching the Service's label selector.

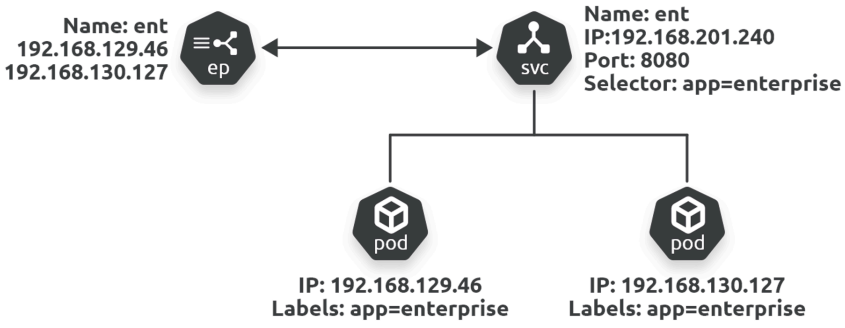

Figure 9.3

The kubelet agent on every node is watching the API server for new Endpoints/EndpointSlice objects. When it sees one, it creates local networking rules to redirect ClusterIP traffic to Pod IPs. In modern Linux-based Kubernetes clusters, the technology used to create these rules is the Linux IP Virtual Server (IPVS). Older versions used iptables.

At this point the Service is fully registered and ready to be used:

- Its front-end configuration is registered with DNS
- Its back-end label selector is created
- Its Endpoints object (or EndpointSlice) is created
- Nodes and `kube-proxies` have created the necessary local routing rules

Summarising service registration

Let's summarise the service registration process with the help of a simple flow diagram.

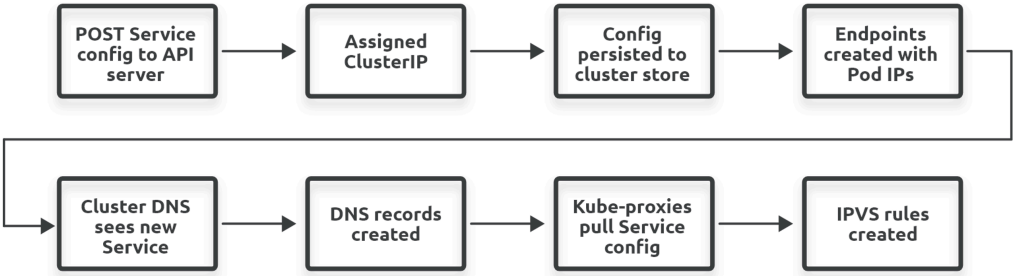

Figure 9.4

You post a new Service resource manifest to the API server where it's authenticated and authorized. The Service is allocated a ClusterIP and its configuration is persisted to the cluster store. An associated Endpoints or EndpointSlice object is created to hold the list of healthy Pod IPs matching the label selector. The cluster DNS is running as a Kubernetes-native application and watching the API server for new Service objects. It observes it and registers the appropriate DNS A and SRV records. Every node is running a kube-proxy that observes the new objects and creates local IPVS/iptables rules so traffic to the Service's ClusterIP is routed to the Pods matching the Service's label selector.

Service discovery

Let's assume there are two microservices apps on the same Kubernetes cluster – `enterprise` and `cerritos`. The Pods for `enterprise` sit behind a Service called `ent` and the Pods for `cerritos` sit behind another Service called `cer`. They've been assigned ClusterIPs, registered with DNS, and things are as follows.

App	Service name	ClusterIP
Enterprise	ent	192.168.201.240
Cerritos	cer	192.168.200.217

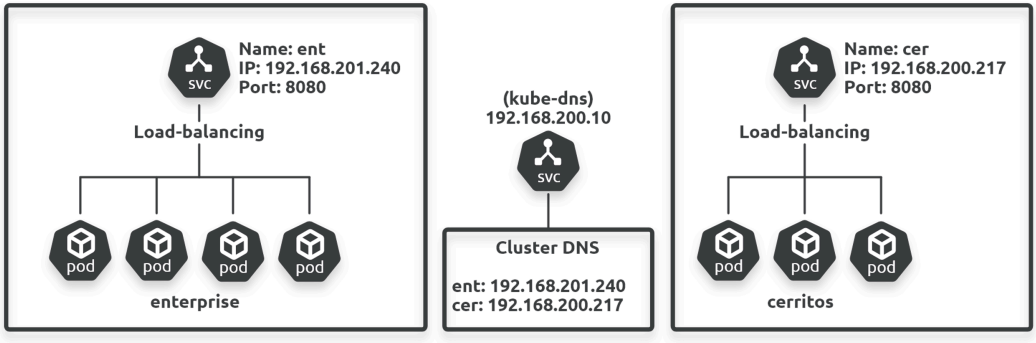

Figure 9.5

For service discovery to work, apps need to know both of the following.

1. The **name** of the other apps they want to connect to (the Service fronting it)
2. How to convert the **name** to an IP address

Application developers are responsible for point 1. They need to code apps with the names of other apps they want to consume. Actually, they need to code the names of Services fronting the remote apps. For example, if the `cerritos` app wants to connect to `enterprise`, it needs to know the name of the `ent` Service.

Kubernetes takes care of point 2, converting the name to an IP.

Converting names to IP addresses using the cluster DNS

Kubernetes automatically configures every container so it can find and use the cluster DNS to convert Service names to IPs. It does this by populating every container's `/etc/resolv.conf` file with the IP address of cluster DNS Service as well as any search domains that should be appended to unqualified names.

An "unqualified name" is a short name such as `ent`. Appending a search domain converts it to a fully qualified domain name (FQDN) such as `ent.default.svc.cluster.local`.

The following snippet shows a container that is configured to send DNS queries to the cluster DNS at 192.168.200.10. It also lists three search domains to append to unqualified names.

```
$ cat /etc/resolv.conf
search svc.cluster.local cluster.local default.svc.cluster.local
nameserver 192.168.200.10
options ndots:5
```

The following snippet proves that `nameserver` in the previous `/etc/resolv.conf` matches the IP address of the cluster DNS (the kube-dns Service). This means Service names will be sent to the cluster DNS for conversion to IP addresses.

```
$ kubectl get svc -n kube-system -l k8s-app=kube-dns
NAME       TYPE        CLUSTER-IP          PORT(S)                  AGE
kube-dns   ClusterIP   192.168.200.10      53/UDP,53/TCP,9153/TCP   3h53m
```

With what you've learned so far, let's talk through the process of the `enterprise` app sending connections to the `cerritos` app.

The process is as follows.

1. Know the name of the remote app (Service)
2. Name resolution
3. Network routing

First up, the `enterprise` app needs to know to send connections to the `cer` Service. That's the job of the app developer. However, once it knows the remote Service name, it needs to convert it into an IP address. Fortunately, the container it's running in knows how to do this, and sends it to the cluster DNS where it's resolved to a ClusterIP (in this case, 192.168.200.217).

All good so far, but ClusterIPs are virtual IPs that require additional magic before traffic reaches the `cerritos` app.

Some network magic

ClusterIPs are on a *"special"* network called the *service network*, and there are no routes to it! This means containers send all ClusterIP traffic to their *default gateway*.

> **Terminology**: A *default gateway* is where devices send traffic when there's no known route. Normally, the *default gateway* forwards traffic to another device with a larger routing table in the hope it will have a route to the destination. A simple analogy might be driving from City A to City Z. The local roads in City A probably don't have signposts to City Z, so you follow signs to the major highway/motorway. Once on the highway/motorway there's more chance you'll find directions to City Z. If the first signpost doesn't have a route, you keep driving until you see one that does. Occasionally you don't find a sign and you get lost or run out of battery/fuel. Routing is similar, if a device doesn't have a route for the destination network, it sends it from one default gateway to the next until hopefully a device has a route. As with driving, it's also possible you never find a route and the traffic times out.

The container's default gateway sends the traffic to the node it's running on.

The node doesn't have a route to the *service network* either, so it sends it to its own default gateway. Doing this causes the traffic to be processed by the node's kernel, which is where the magic happens…

Every Kubernetes node runs a system service called `kube-proxy`. At a high-level, `kube-proxy` is responsible for capturing traffic destined for ClusterIPs and redirecting it to the IP addresses of Pods matching the Service's label selector. Let's look a bit closer…

`kube-proxy` is a Pod-based Kubernetes-native app that implements a controller watching the API server for new Services and Endpoints objects. When it sees them, it creates local IPVS rules telling the node to intercept traffic destined for the Service's ClusterIP and forward it to individual Pod IPs.

This means that every time a node's kernel processes traffic headed for an address on the *service network*, a trap occurs, and the traffic is redirected to the IP of a healthy Pod matching the Service's label selector.

Kubernetes originally used iptables to do this trapping and load-balancing. However, it was replaced by IPVS in Kubernetes 1.11. The is because IPVS is a high-performance kernel-based L4 load-balancer that scales better than iptables and implements better load-balancing.

Summarising service discovery

Let's quickly summarise the service discovery process with the help of the flow diagram in Figure 9.6.

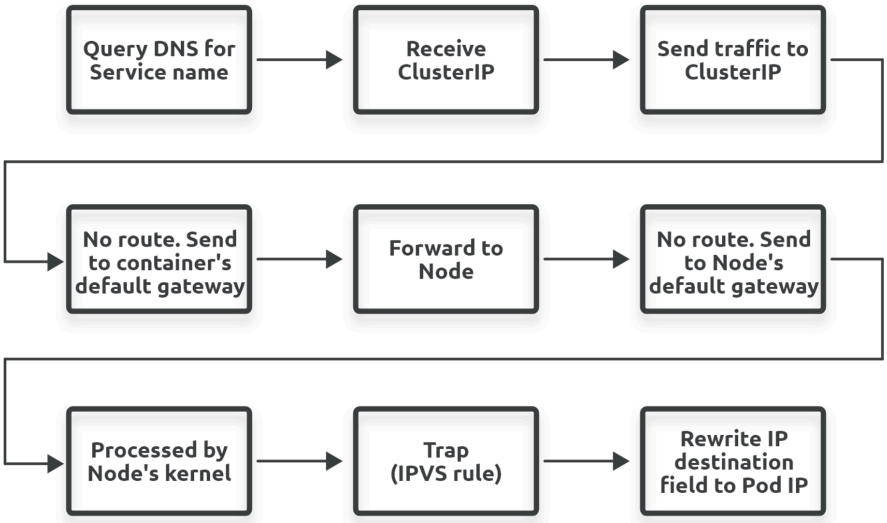

Figure 9.6

Assume the "enterprise" app is sending traffic to "cerritos". First up, it needs the name of the "cer" Service sitting in front of "cerritos". Like we said before, it's the responsibility of the application developer to make sure this is known.

An instance of the "enterprise" app tries to send traffic to the "cer" Service. But networks work with numbers, not names. So the container hosting the "enterprise" app sends the "cer" name to cluster DNS asking it to resolve it to an IP address. The container knows how to do this because it's pre-configured with the address of cluster DNS in its `/etc/resolv.conf` file. The cluster DNS replies with the ClusterIP and the "enterprise" container sends the traffic to the network. However, ClusterIPs are on the special *service network* and the container doesn't have a route to it. So it sends it to its default gateway, which forwards it to the node its running on. The node doesn't have a route either, so it sends it to its own default gateway. En-route, the request is processed by the node's kernel. A trap is triggered, and the request is redirected to the IP address of a Pod that matches the Services label selector.

Service discovery and Namespaces

It's important to understand that every cluster has an *address space*, and that Namespaces partition it.

Cluster *address spaces* are based on a DNS domain that we call the *cluster domain*. The domain name is usually `cluster.local` and objects have unique names within it. For example, a Service called `ent` will have a fully qualified domain name (FQDN) of `ent.default.svc.cluster.local`

The format is `<object-name>.<namespace>.svc.cluster.local`

Namespaces let you partition the address space below the cluster domain. For example, creating a couple of Namespaces called `dev` and `prod` will give you two new address spaces.

- dev: <object-name>.dev.svc.cluster.local

- prod: <object-name>.prod.svc.cluster.local

Object names have to be unique *within* a Namespace but not *across* Namespaces. For example, you can't have two Services called "ent" in the same Namespace, but you can if they're in different Namespaces. This is useful for parallel development and production configurations. For example, Figure 9.7 shows a single cluster divided into `dev` and `prod` Namespaces with identical configurations deployed to each.

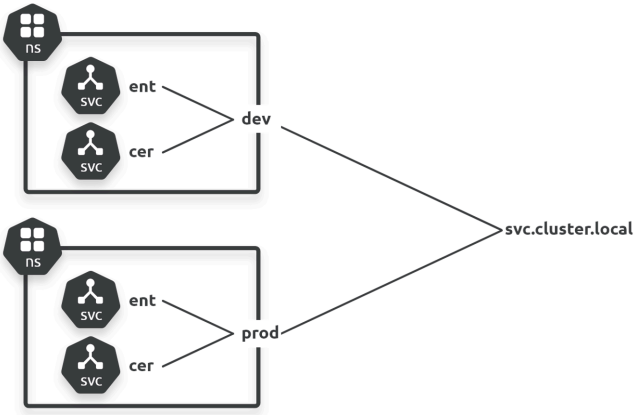

Figure 9.7

Objects can connect to Services in the local Namespace using short names such as `ent` and `cer`. But connecting to objects in a remote Namespace requires FQDNs such as `ent.dev.svc.cluster.local` and `cer.dev.svc.cluster.local`.

Service discovery example

Let's walk through a quick example.

The following YAML is called `sd-example.yml` and it's in the `service-discovery` folder of the book's GitHub repo. It defines two Namespaces, two Deployments, two Services, and a standalone jump Pod. The two Deployments have identical names, as do the Services. However, they're deployed to different Namespaces. The jump Pod is deployed to the `dev` Namespace.

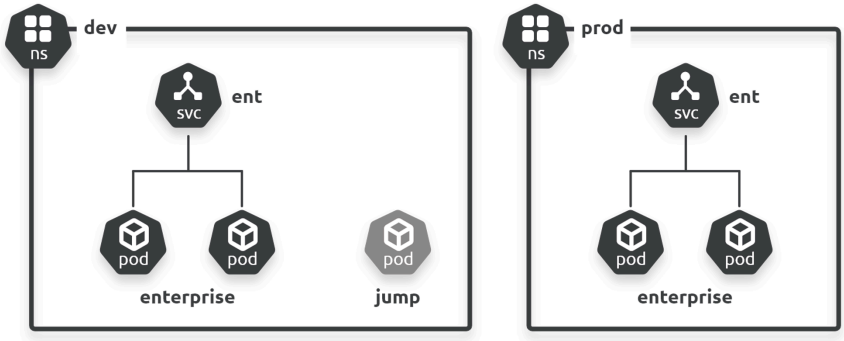

Figure 9.8

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: v1
kind: Namespace
metadata:
  name: prod
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: enterprise
  labels:
    app: enterprise
  namespace: dev
spec:
  selector:
    matchLabels:
      app: enterprise
  replicas: 2
  template:
    metadata:
      labels:
        app: enterprise
    spec:
      containers:
      - image: nigelpoulton/k8sbook:text-dev
        name: enterprise-ctr
        ports:
        - containerPort: 8080
---
apiVersion: apps/v1
kind: Deployment
```

```
metadata:
  name: enterprise
  labels:
    app: enterprise
  namespace: prod
spec:
  selector:
    matchLabels:
      app: enterprise
  replicas: 2
  template:
    metadata:
      labels:
        app: enterprise
    spec:
      containers:
      - image: nigelpoulton/k8sbook:text-prod
        name: enterprise-ctr
        ports:
        - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: ent
  namespace: dev
spec:
  selector:
    app: enterprise
  ports:
    - port: 8080
  type: ClusterIP
---
apiVersion: v1
kind: Service
metadata:
  name: ent
  namespace: prod
spec:
  selector:
    app: enterprise
  ports:
    - port: 8080
  type: ClusterIP
---
apiVersion: v1
kind: Pod
metadata:
  name: jump
  namespace: dev
spec:
```

```
  terminationGracePeriodSeconds: 5
  containers:
  - name: jump
    image: ubuntu
    tty: true
    stdin: true
```

Deploy the configuration to your cluster.

```
$ kubectl apply -f sd-example.yml
namespace/dev created
namespace/prod created
deployment.apps/enterprise created
deployment.apps/enterprise created
service/ent created
service/ent created
pod/jump-pod created
```

Check it was correctly applied. The following outputs are trimmed to fit the page and don't show all objects.

```
$ kubectl get all --namespace dev
NAME          TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
service/ent   ClusterIP   192.168.202.57   <none>        8080/TCP   43s

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/enterprise   2/2     2            2           43s
<snip>

$ kubectl get all --namespace prod
NAME          TYPE        CLUSTER-IP        EXTERNAL-IP   PORT(S)    AGE
service/ent   ClusterIP   192.168.203.158   <none>        8080/TCP   52s

NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/enterprise   2/2     2            2           52s
<snip>
```

You have an `enterprise` app and `ent` Service in both Namespaces (dev and prod). You also have a jump Pod in the dev Namespace. Let's test how service discovery works within a Namespace and across them.

The next steps will:

1. Log on to the jump Pod in the `dev` Namespace
2. Check its `/etc/resolv.conf` file
3. Connect to `ent` in the local `dev` Namespace
4. Connect to `ent` in the remote `prod` Namespace

To help with the demo, the versions of the app used in each Namespace are slightly different.

Log on to the jump Pod.

```
$ kubectl exec -it jump --namespace dev -- bash
root@jump:/#
```

Your terminal prompt will change to indicate you are attached to the jump Pod.

Inspect the contents of the `/etc/resolv.conf` file and check the search domains listed include the `dev` Namespace (`search dev.svc.cluster.local`) and not the `prod` Namespace.

```
$ cat /etc/resolv.conf
search dev.svc.cluster.local svc.cluster.local cluster.local default.svc.cluster.local
nameserver 192.168.200.10
options ndots:5
```

The search domains lists the `dev` Namespace, and the `nameserver` is set to the IP of the cluster DNS.

Install the `curl` utility.

```
$ apt-get update && apt-get install curl -y
<snip>
```

Use `curl` to connect to the version of the app running in `dev` by using the `ent` short name. The app listens on port `8080`.

```
$ curl ent:8080
Hello from the DEV Namespace!
Hostname: enterprise-7d49557d8d-k4jjz
```

The "Hello from the DEV Namespace" response proves the connection reached the instance in the `dev` Namespace.

When the `curl` command was issued, the container automatically appended `dev.svc.cluster.local` to the `ent` name and sent the query to the cluster DNS specified in `/etc/resolv.conf`. DNS returned the ClusterIP for the `ent` Service in the local `dev` Namespace and the app sent the traffic to that address. En-route to the node's default gateway the traffic triggered a trap in the node's kernel and was redirected to one of the Pods hosting the app.

Run the `curl` command again, but this time append the domain name of the `prod` Namespace. This will cause the cluster DNS to return the ClusterIP for the instance in the `prod` Namespace and traffic will eventually reach a Pod running in `prod`

```
$ curl ent.prod.svc.cluster.local:8080
Hello from the PROD Namespace!
Hostname: enterprise-5464d8c4f9-v7xsk
```

This time the response comes from a Pod in the `prod` Namespace.

The test proves that short names are resolved to the local Namespace (the same Namespace the app is running in) and connecting across Namespaces requires FQDNs.

Remember to detach your terminal from the container by typing `exit`.

Troubleshooting service discovery

Service registration and discovery involves a lot of moving parts. If any of them stops working, the whole process can break. Let's quickly run through what needs to be working and how to check them.

Kubernetes uses the cluster DNS as its *service registry*. This runs as one or more Pods in the `kube-system` Namespace with a Service object providing a stable endpoint. The important components are:

- Pods: Managed by the `coredns` Deployment
- Service: A ClusterIP Service called `kube-dns` listening on port 53 TCP/UDP
- Endpoints object: Also called `kube-dns`

All objects relating to the cluster DNS are in the `kube-system` Namespace and tagged with the `k8s-app=kube-dns` label. This is helpful when filtering `kubectl` output.

Make sure the `coredns` Deployment and its managed Pods are up and running.

```
$ kubectl get deploy -n kube-system -l k8s-app=kube-dns
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
coredns   2/2     2            2           28d

$ kubectl get pods -n kube-system -l k8s-app=kube-dns
NAME                       READY   STATUS    RESTARTS   AGE
coredns-5644d7b6d9-74pv7   1/1     Running   0          28d
coredns-5644d7b6d9-s759f   1/1     Running   0          28d
```

Check the logs from each of the `coredns` Pods. You'll need to substitute the names of the Pods in your environment. The following output is typical of a working DNS Pod.

```
$ kubectl logs coredns-5644d7b6d9-74pv7 -n kube-system
2020-02-19T21:31:01.456Z [INFO] plugin/reload: Running configuration...
2020-02-19T21:31:01.457Z [INFO] CoreDNS-1.6.2
2020-02-19T21:31:01.457Z [INFO] linux/amd64, go1.12.8, 795a3eb
CoreDNS-1.6.2
linux/amd64, go1.12.8, 795a3eb
```

Assuming the Pods and Deployment are working, you should also check the Service and associated Endpoints object. The output should show the service is up, has an IP address in the ClusterIP field, and is listening on port 53 TCP/UDP.

The ClusterIP address for the `kube-dns` Service should match the IP address in the `/etc/resolv.conf` files of all containers on the cluster. If the IP addresses are different, containers will send DNS requests to the wrong IP address.

```
$ kubectl get svc kube-dns -n kube-system
NAME       TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)                  AGE
kube-dns   ClusterIP   192.168.200.10   <none>        53/UDP,53/TCP,9153/TCP   28d
```

The associated `kube-dns` Endpoints object should also be up and have the IP addresses of the `coredns` Pods listening on port 53.

```
$ kubectl get ep -n kube-system -l k8s-app=kube-dns
NAME       ENDPOINTS                                                     AGE
kube-dns   192.168.128.24:53,192.168.128.3:53,192.168.128.24:53 + 3 more...   28d
```

Once you've verified the fundamental DNS components are up and working, you can proceed to perform more detailed and in-depth troubleshooting. Here are some simple tips.

Start a troubleshooting Pod that has your favourite networking tools installed (ping, traceroute, curl, dig, nslookup etc.). The standard `gcr.io/kubernetes-e2e-test-images/dnsutils:1.3` image is a popular choice if you don't have your own custom image with your tools installed. Unfortunately, there's no image tagged as `latest` in the repo. This means you have to specify a version. At the time of writing, 1.3 has been the latest version for a long time.

The following command starts a new standalone Pod called `netutils`, based on the dnsutils image just mentioned. It will also connect your terminal to it.

```
$ kubectl run -it dnsutils \
  --image gcr.io/kubernetes-e2e-test-images/dnsutils:1.3
```

A common way to test DNS resolution is to use `nslookup` to resolve the `kubernetes` Service fronting the API server. The query should return an IP address as well as the name `kubernetes.default.svc.cluster.local`.

```
# nslookup kubernetes
Server:         192.168.200.10
Address:        192.168.200.10#53

Name:   kubernetes.default.svc.cluster.local
Address: 192.168.200.1
```

The first two lines should return the IP address of your cluster DNS. The last two lines should show the FQDN of the `kubernetes` Service and its ClusterIP. You can verify the ClusterIP of the `kubernetes` Service by running a `kubectl get svc kubernetes` command.

Errors such as "nslookup: can't resolve kubernetes" are possible indicators that DNS isn't working. A possible solution is to restart the `coredns` Pods. They're managed by a Deployment object and will be automatically recreated.

The following command deletes the DNS Pods and must be ran from a terminal with `kubectl` installed. If you're still logged on to the `netutils` Pod, you'll need to type `exit` to disconnect.

```
$ kubectl delete pod -n kube-system -l k8s-app=kube-dns
pod "coredns-5644d7b6d9-2pdmd" deleted
pod "coredns-5644d7b6d9-wsjzp" deleted
```

Verify they've restarted and test DNS again. They'll be restarted because they're managed by a Deployment object.

Summary

In this chapter, you learned that Kubernetes uses the internal cluster DNS for service registration and service discovery.

All new Service objects are automatically registered with the cluster DNS, and all containers are pre-configured to use the cluster DNS for service discovery.

The cluster DNS resolves Service names to ClusterIPs. These are stable virtual IPs on a special network called the *service network*. Although there are no routes to this network, the `kube-proxy` configures all cluster nodes to redirect traffic destined for the service network to Pod IPs on the Pod network.